

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool: Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand

9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply
10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply

Code of Conduct:

I K. Sampath Chaitanya (192572006) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K. Sampath

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	Produces clean, well-structured code with proper comments and	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1) Create Table Debugging

Incorrect code:

```
mysql> CREATE TABLE Employee
-> (
-> EmpID INT
-> EmpName VARCHAR(30),
-> Salary INT
-> );
ERROR 1064 (42000): You have an error in your SQL syntax; check
the manual that corresponds to your MySQL server version for the
right syntax to use near 'EmpName VARCHAR(30),
Salary INT
)' at line 4
```

Error Identified:

- The CREATE TABLE statement contained syntax errors and missing constraints such as PRIMARY KEY and NOT NULL.

Correct code:

```
mysql> CREATE TABLE Employee
-> (
-> EmpID INT PRIMARY KEY,
-> EmpName VARCHAR(30) NOT NULL,
-> Salary INT
-> );
Query OK, 0 rows affected (0.02 sec)
```

The syntax errors were fixed and the required constraints were added. The table was created successfully.

```
mysql> DESC Employee;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| EmpID | int           | NO   | PRI | NULL    |       |
| EmpName | varchar(30)   | NO   |     | NULL    |       |
| Salary | int           | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.02 sec)
```

2) Aggregate Function Debugging

```
mysql> SELECT Salary,COUNT(*)
-> FROM Employee;
ERROR 1140 (42000): In aggregated query without GROUP BY, expres
sion #1 of SELECT list contains nonaggregated column 'co2_at3.Em
ployee.Salary'; this is incompatible with sql_mode=only_full_gro
up_by
```

```
mysql> SELECT Salary,
-> COUNT(*) AS Total
-> FROM Employee
-> GROUP BY Salary;
+-----+-----+
| Salary | Total |
+-----+-----+
| 25000 | 1 |
| 35000 | 1 |
| 28000 | 1 |
+-----+-----+
3 rows in set (0.00 sec)
```

3) Column Name Correction

```
mysql> SELECT StudntName
-> FROM Student;
ERROR 1054 (42S22): Unknown column 'StudntName' in 'field list'
mysql> |
```

Incorrect Query:

The query contains a misspelled column name, causing an unknown column error.

```
mysql> SELECT StudentName
-> FROM Student;
```

```
+-----+
| StudentName |
+-----+
| Arun        |
| Priya       |
| Rahul       |
| Divya       |
| Karthik     |
+-----+
```

```
5 rows in set (0.00 sec)
```

Corrected Query:

The column name was corrected, and the query executed successfully.

4) JOIN Query Correction

```
mysql> SELECT *
-> FROM Student
-> JOIN Department;
```

```
+-----+-----+-----+-----+-----+-----+
| RegNo | StudentName | Age | DeptID | DeptID | DeptName |
+-----+-----+-----+-----+-----+-----+
| 101   | Arun        | 20  | 1      | 3      | EEE      |
| 101   | Arun        | 20  | 1      | 2      | ECE      |
| 101   | Arun        | 20  | 1      | 1      | CSE      |
| 102   | Priya       | 19  | 2      | 3      | EEE      |
| 102   | Priya       | 19  | 2      | 2      | ECE      |
| 102   | Priya       | 19  | 2      | 1      | CSE      |
| 103   | Rahul       | 21  | 1      | 3      | EEE      |
| 103   | Rahul       | 21  | 1      | 2      | ECE      |
| 103   | Rahul       | 21  | 1      | 1      | CSE      |
| 104   | Divya       | 20  | 3      | 3      | EEE      |
| 104   | Divya       | 20  | 3      | 2      | ECE      |
| 104   | Divya       | 20  | 3      | 1      | CSE      |
| 105   | Karthik     | 22  | 2      | 3      | EEE      |
| 105   | Karthik     | 22  | 2      | 2      | ECE      |
| 105   | Karthik     | 22  | 2      | 1      | CSE      |
+-----+-----+-----+-----+-----+-----+
```

```
15 rows in set (0.00 sec)
```

Incorrect Query:

The JOIN statement is missing the join condition, resulting in an incorrect query.

```
mysql> SELECT Student.StudentName,
-> Department.DeptName
-> FROM Student
-> JOIN Department
-> ON Student.DeptID=Department.DeptID;
```

StudentName	DeptName
Arun	CSE
Rahul	CSE
Priya	ECE
Karthik	ECE
Divya	EEE

5 rows in set (0.00 sec)

Corrected Query:

The correct ON condition was added, and the tables were joined successfully

5) Subquery Correction

```
mysql> SELECT *
-> FROM Student
-> WHERE DeptID=
-> (
-> SELECT DeptID
-> FROM Department
-> );
ERROR 1242 (21000): Subquery returns more than 1 row
```

Incorrect Query:

The subquery returns multiple rows where only a single value is expected.

```
mysql> SELECT *
-> FROM Student
-> WHERE DeptID IN
-> (
-> SELECT DeptID
-> FROM Department
-> );
```

RegNo	StudentName	Age	DeptID
101	Arun	20	1
103	Rahul	21	1
102	Priya	19	2
105	Karthik	22	2
104	Divya	20	3

5 rows in set (0.00 sec)

Corrected Query:

The IN operator was used to handle multiple values, and the query executed successfully.

6) VIEW Debugging

```
mysql> CREATE VIEW StudentView AS
-> SELECT StudentNam
-> FROM Student;
ERROR 1054 (42S22): Unknown column 'StudentNam' in 'field list'
```

Incorrect Query:

The VIEW creation failed because the view already existed (or contained an invalid reference).

```
mysql> SELECT * FROM StudentView;
+-----+
| StudentName |
+-----+
| Arun        |
| Priya       |
| Rahul       |
| Divya       |
| Karthik     |
+-----+
5 rows in set (0.00 sec)
```

Corrected Query:

The existing view was removed and recreated successfully with the correct SQL statement.

7) Query Optimization

Original Query:

The query contains redundant conditions, making it unnecessarily complex.

```
mysql> SELECT *
-> FROM Student
-> WHERE Age>18
-> AND Age>19;
+-----+-----+-----+-----+
| RegNo | StudentName | Age | DeptID |
+-----+-----+-----+-----+
| 101   | Arun        | 20  | 1       |
| 103   | Rahul       | 21  | 1       |
| 104   | Divya       | 20  | 3       |
| 105   | Karthik     | 22  | 2       |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Optimized Query:

The redundant condition was removed, producing the same result with a simpler and more efficient query.

```
mysql> SELECT *
-> FROM Student
-> WHERE Age>19;
+-----+-----+-----+-----+
| RegNo | StudentName | Age | DeptID |
+-----+-----+-----+-----+
| 101   | Arun        | 20  | 1       |
| 103   | Rahul       | 21  | 1       |
| 104   | Divya       | 20  | 3       |
| 105   | Karthik     | 22  | 2       |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Same output, but the optimized query is simpler because it removes the redundant condition.

8) Integrity Constraint Correction

```
mysql> INSERT INTO Student2
-> VALUES
-> (101, NULL, 15);
ERROR 1048 (23000): Column 'StudentName' cannot be null
```

Incorrect Data:

The inserted data violates one or more integrity constraints, causing the insertion to fail.

```
mysql> INSERT INTO Student2
-> VALUES
-> (101, 'Meena', 19);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT *
-> FROM Student2;
+-----+-----+-----+
| RegNo | StudentName | Age |
+-----+-----+-----+
| 101   | Meena       | 19  |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Corrected Data:

The data was modified to satisfy all integrity constraints and was inserted successfully.

9) 1NF / 2NF Normalization

Original relation containing repeating groups (violates 1NF).


```
mysql> SELECT * FROM StudentCourse;
+-----+-----+-----+-----+
| RegNo | StudentName | Course1 | Course2 |
+-----+-----+-----+-----+
| 101   | Arun        | DBMS    | Java    |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Creation of normalized tables.

```
mysql> CREATE TABLE StudentNew
-> (
-> RegNo INT PRIMARY KEY,
-> StudentName VARCHAR(30)
-> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE Registration
-> (
-> RegNo INT,
-> CourseName VARCHAR(30),
-> PRIMARY KEY(RegNo,CourseName)
-> );
Query OK, 0 rows affected (0.01 sec)
```

Data inserted into normalized tables.

```
mysql> INSERT INTO StudentNew VALUES
-> (101,'Arun');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Registration VALUES
-> (101,'DBMS'),
-> (101,'Java');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

Normalized tables after removing repeating groups (1NF achieved)

```
mysql> SELECT * FROM StudentNew;
+-----+-----+
| RegNo | StudentName |
+-----+-----+
| 101   | Arun        |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM Registration;
+-----+-----+
| RegNo | CourseName |
+-----+-----+
| 101   | DBMS       |
| 101   | Java       |
+-----+-----+
2 rows in set (0.00 sec)
```

Original Relation:

The table contains repeating groups, violating the First Normal Form (1NF).

Below Normalized Tables Screenshot

Normalized Relation:

The relation was decomposed into separate tables, eliminating repeating groups and improving data organization.

10) 3NF Normalization

Creating the Department table to store department details separately.

```
mysql> CREATE TABLE DepartmentNew
-> (
-> DeptID INT PRIMARY KEY,
-> DeptName VARCHAR(30),
-> DeptPhone VARCHAR(15)
-> );
Query OK, 0 rows affected (0.02 sec)
```

Creating the Student table with a foreign key reference.

```
mysql> CREATE TABLE StudentFinal
-> (
-> RegNo INT PRIMARY KEY,
-> StudentName VARCHAR(30),
-> DeptID INT,
-> FOREIGN KEY(DeptID)
-> REFERENCES DepartmentNew(DeptID)
-> );
Query OK, 0 rows affected (0.03 sec)
```

Inserting data into the normalized tables.

```
mysql> INSERT INTO DepartmentNew VALUES  
-> (1,'CSE','9876543210'),  
-> (2,'ECE','9876500000');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO StudentFinal VALUES  
-> (101,'Arun',1),  
-> (102,'Priya',2);  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0
```

Normalized tables after applying Third Normal Form (3NF).

```
mysql> select * from studentfinal;  
+-----+-----+-----+  
| RegNo | StudentName | DeptID |  
+-----+-----+-----+  
| 101 | Arun | 1 |  
| 102 | Priya | 2 |  
+-----+-----+-----+  
2 rows in set (0.00 sec)  
  
mysql> select * from DepartmentNew;  
+-----+-----+-----+  
| DeptID | DeptName | DeptPhone |  
+-----+-----+-----+  
| 1 | CSE | 9876543210 |  
| 2 | ECE | 9876500000 |  
+-----+-----+-----+  
2 rows in set (0.00 sec)
```